

DriverManager VS DataSource

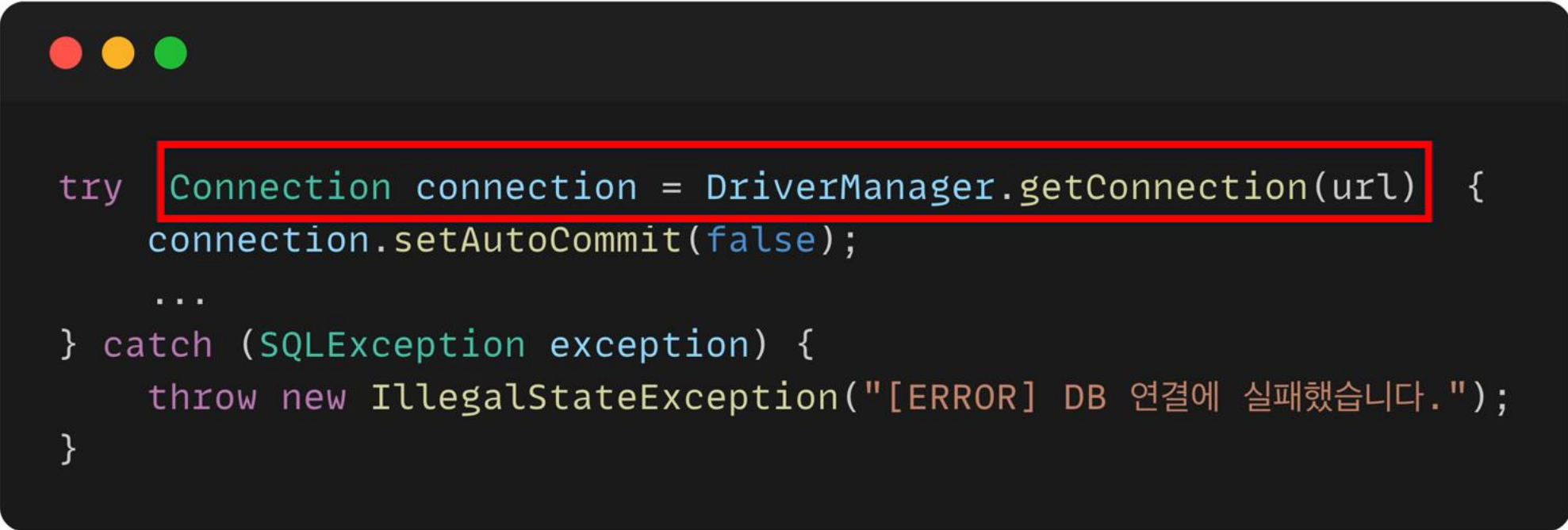
DB 연결 방식의 차이와 진화



```
Connection c1 = DriverManager.getConnection(url, user, password);  
Connection c2 = dataSource.getConnection();
```

대상 독자

JDBC를 써본 적은 있지만
DriverManager/DataSource 내부는 모르는 사람

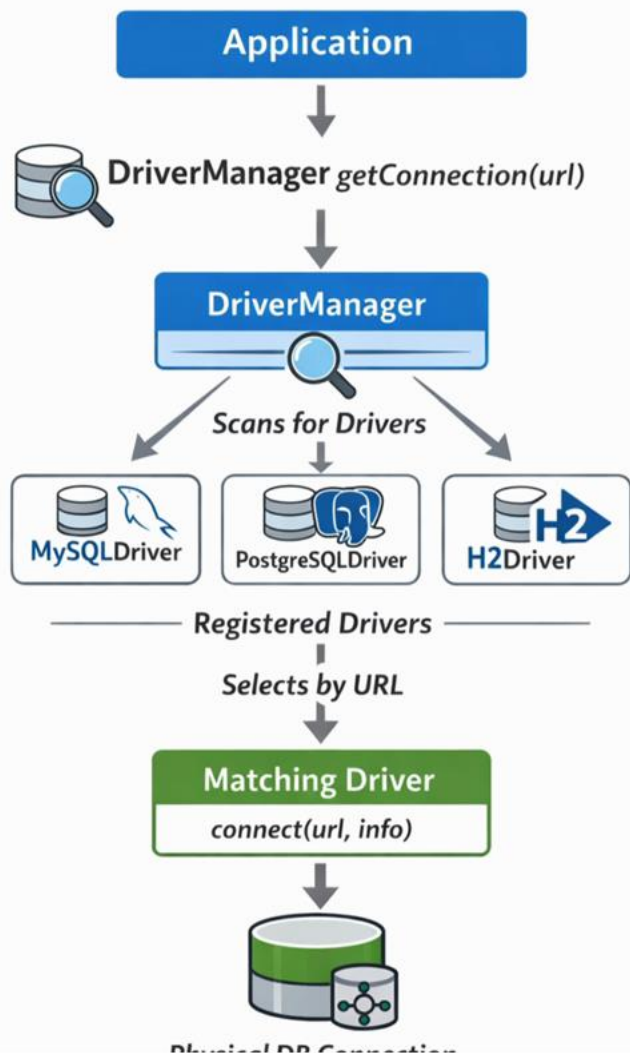


```
try Connection connection = DriverManager.getConnection(url) {  
    connection.setAutoCommit(false);  
    ...  
} catch (SQLException exception) {  
    throw new IllegalStateException("[ERROR] DB 연결에 실패했습니다.");  
}
```

DriverManager란?

JDBC Driver를 관리하고,
URL에 맞는 Driver를 찾아 연결을 얻도록 돕는 클래스
실제 Connection 생성은 선택된 Driver가 수행한다

DriverManager는 등록된 Driver를 순회하며 위임한다



```
for (DriverInfo aDriver : registeredDrivers) {  
    Connection con = aDriver.driver.connect(url, info);  
    if (con != null) {  
        return con;  
    }  
}
```

H2 Driver

```
@Override
public Connection connect(String url, Properties info) throws SQLException {
    if (url == null) {
        throw DbException.getJdbcSQLException(ErrorCode.URL_FORMAT_ERROR_2, null, Constants.URL_FORMAT, null);
    } else if (url.startsWith(Constants.START_URL)) {
        return new JdbcConnection(url, info, null, null, false);
    } else if (url.equals(DEFAULT_URL)) {
        return DEFAULT_CONNECTION.get();
    } else {
        return null;
    }
}
```

```
public static final String START_URL = "jdbc:h2:";
```

DriverManager의 한계

DriverManager를 호출부에서 직접 쓰면
저수준 연결 획득 방식에 결합될 수 있다.

DriverManager의 한계

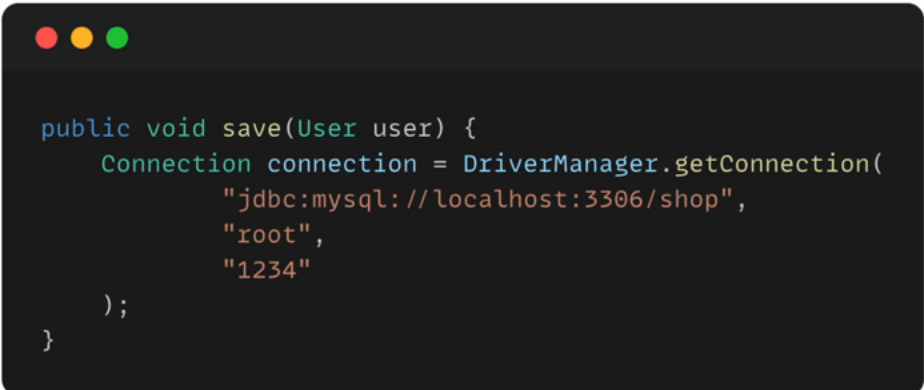
```
public void save(User user) {  
    Connection connection = DriverManager.getConnection(  
        "jdbc:mysql://localhost:3306/shop",  
        "root",  
        "1234"  
    );  
}
```

호출하는 코드가 DB 접속 정보까지 알고 있어야 한다

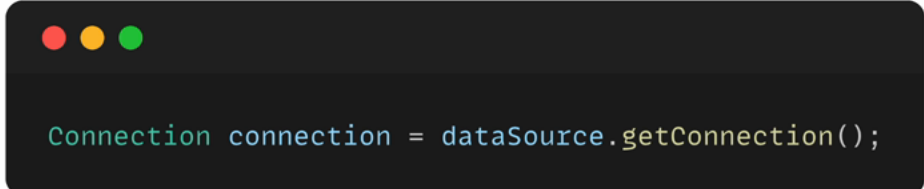
Connection을 어떻게 만들지에 대한 책임이 호출부 안에 들어와 있다

DataSource의 등장

애플리케이션 규모와 요구사항이 커지면서
Connection 획득 책임을 분리하고 추상화 시킬 필요가 생겼다



```
public void save(User user) {  
    Connection connection = DriverManager.getConnection(  
        "jdbc:mysql://localhost:3306/shop",  
        "root",  
        "1234"  
    );  
}
```



```
Connection connection = dataSource.getConnection();
```

애플리케이션은 “어떻게 연결되는지”가 아니라
“연결을 달라”는 메시지만 보낸다.

DataSource란?

Interface DataSource

All Superinterfaces:

`CommonDataSource`, `Wrapper`

```
public interface DataSource
extends CommonDataSource, Wrapper
```

A factory for connections to the physical data source that this DataSource object represents. An alternative to the DriverManager facility, a DataSource object is the preferred means of getting a connection. An object that implements the DataSource interface will typically be registered with a naming service based on the Java Naming and Directory (JNDI) API.

A DataSource object has properties that can be modified when necessary. For example, if the data source is moved to a different server, the property for the server can be changed. The benefit is that because the data source's properties can be changed, any code accessing that data source does not need to be changed.

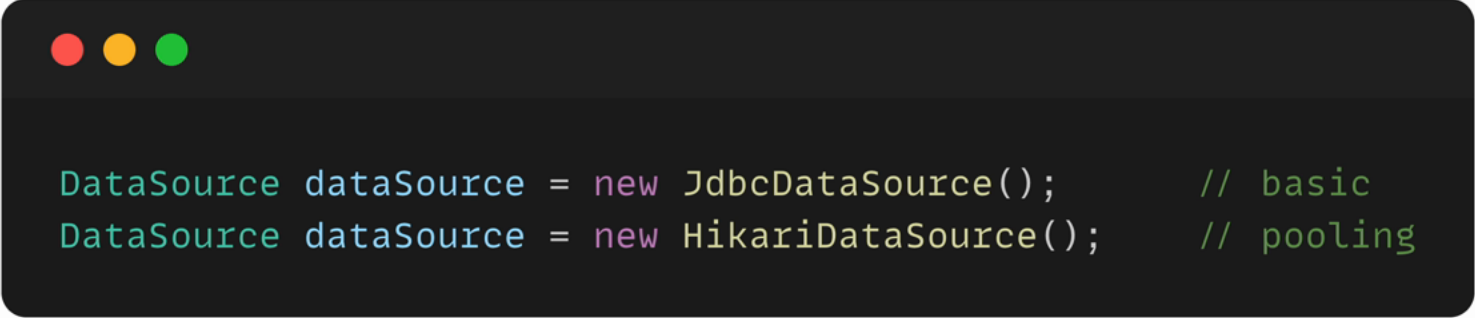
**DataSource는 필요한 연결 설정을 가진,
DriverManager의 대안이자 권장 Connection Factory다**

DataSource = Interface

```
public interface DataSource extends CommonDataSource, Wrapper {  
  
    Connection getConnection() throws SQLException;  
  
    Connection getConnection(String username, String password) throws SQLException;  
}
```

구현은 라이브러리, DB벤더사 등이 담당하며,
실제 연결 설정과 동작은 구현 객체가 가진다

DataSource = Interface



```
DataSource dataSource = new JdbcDataSource();    // basic
DataSource dataSource = new HikariDataSource();  // pooling
```

다형성을 활용한
연결 생성 방식의 추상화

DataSource != Connection Pool

Basic implementation
표준 Connection 생성

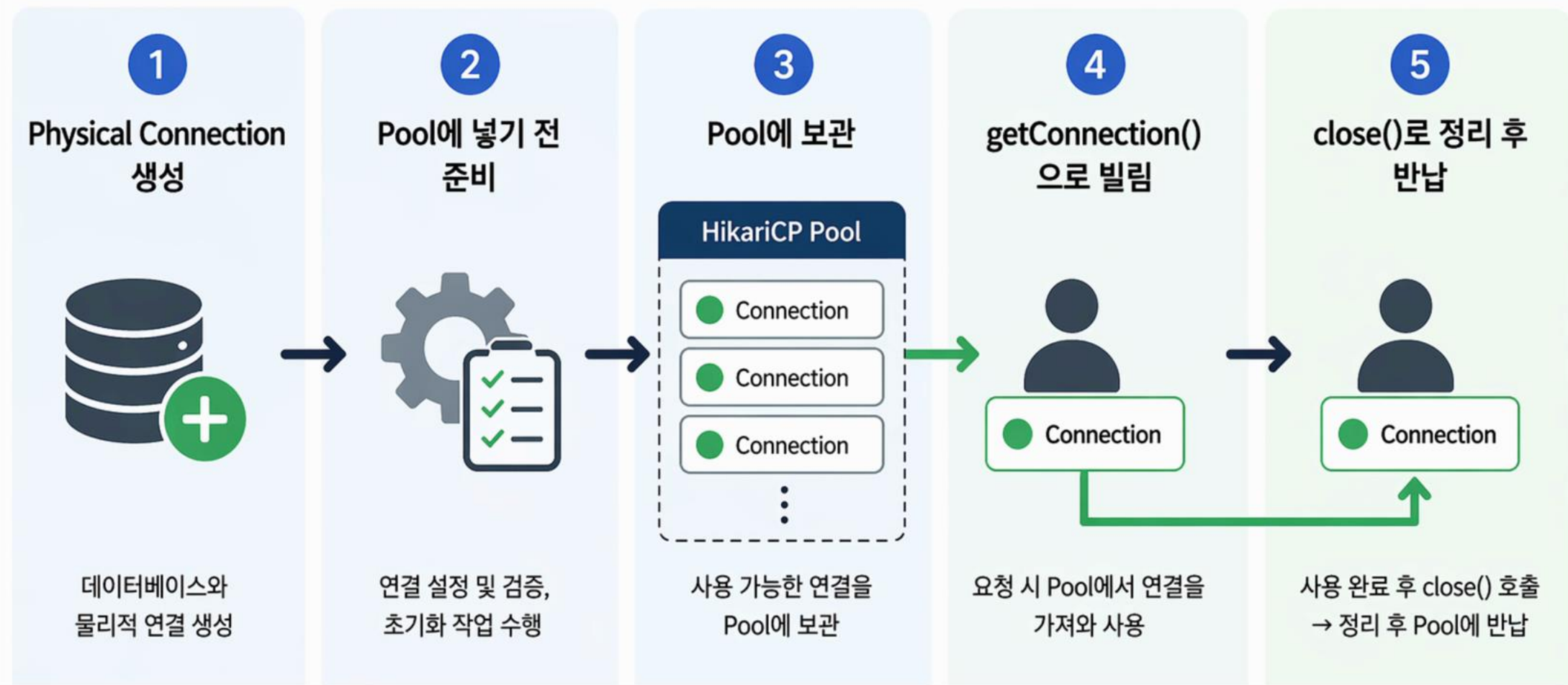
Connection pooling implementation
pool과 함께 동작하는 Connection 생성

Distributed transaction implementation
분산 트랜잭션용 Connection 생성

HikariCP

Connection pooling implementation
pool과 함께 동작하는 Connection 생성

HikariCP 내부 동작



마무리

1. DriverManager

→ Driver를 찾아
직접 Connection 생성



Application

요청



Driver 찾기



새 Connection 생성

2. DataSource

→ Connection 생성
책임을 객체로 분리



Application

요청

DataSource
(Connection 생성 담당)



새 Connection 생성

3. HikariCP

→ Connection을
생성하지 않고 재사용
(Pooling)



Application

요청

HikariCP (Connection Pool)



미리 생성된 Connection 저장



Database

재사용 (반환)